tensormap接口使用说明

参考文档：SiPU 1.5 DTE 架构设计方案 v1.2

NOTES：新接口和老接口的差异

1. 新接口接收一个host的tmap对象地址，根据传参填充tmap结构体
2. tmap和tensorDataType和tensorRank这3个参数类型和老接口保持一致
3. 新接口根据参数globalDimInElements，boxDimInElements自动计算tile dim和supertile dim以及globalStride（输入参数从之前的tile为单位，全部改为以element为单位）
4. oobFillValue参数在driver api中会直接透传给tmap

1. siTensorMapEncodeTiled

1.1 接口说明

代码块

```
/**
 * @brief Encode a tiled tensor map
 * @note this function only support tiled tensor map
 * @param tmap Tensor map object to create.
 * @param tensorDataType Tensor data type
 * @param tensorRank  Dimensionality of tensor; range is [2, 5]
 * @param globalAddress Starting address of memory region described by tensor, 256-byte aligned
 * @param globalDimInElements Array containing tensor size (number of elements) along each of the
 * tensorRank dimensions. eg, {512, 512}. Length of the array is tensorRank.
 * @param boxDimInElements Array containing traversal box size (number of elments) along each of the
 * tensorRank dimensions. Specifies how many elements to be traversed along each tensor dimension.
 * eg, {128, 128}. Length of the array is tensorRank.
```

```
 * @note the dim[0] and dim[1]'s value of boxDimInElements must be divisible
by the tile dimension.
 * @param oobFillValue Value to fill out-of-bound elements
 * @return SIresult
 */
SIresult siTensorMapEncodeTiled(SItensorMap* tmap,
                                SItensorMapDataType tensorDataType,
                                uint8_t tensorRank,
                                uint8_t* globalAddress,
                                uint32_t* globalDimInElements,
                                uint32_t* boxDimInElements,
                                uint32_t oobFillValue);
```

tensormap结构体成员变量命名如下

👍 如果需要修改单个成员，可以访问tensormap结构体的成员变量，修改其数值实现。

代码块

```
    uint8_t tensor_datatype;
    uint8_t tensor_rank;
    uint8_t tensor_tiled;
    uint8_t resv1;
    uint32_t tile_dim[2];
    uint32_t tile_headersize;
    uint32_t tile_datasize;
    uint8_t supertile_dim[2];
    uint8_t resv2[2];
    uint64_t global_addr;
    uint32_t global_dim[5];
    uint32_t global_strides[5];
    uint32_t box_dim[5];
    uint32_t oob_fillvalue;
    uint8_t resv3[32];
```

1.2 参数说明

参数类型	参数名	参数解释	备注
SItensorMap*	tmap	Tensormap host地址	HOST地址
	tensorDataType	tensor的数据类型	enum TmapDtype

SItensorMapDataType			{
			TMAP_DTYPE_UINT4 = 0, // 0
			TMAP_DTYPE_INT4, // 1
			TMAP_DTYPE_UINT8, // 2
			TMAP_DTYPE_INT8, // 3
			TMAP_DTYPE_FP8, // 4
			TMAP_DTYPE_FP16, // 5
			TMAP_DTYPE_BF16, // 6
			TMAP_DTYPE_UINT32, // 7
			TMAP_DTYPE_INT32, // 8
			TMAP_DTYPE_FP32, // 9
			TMAP_DTYPE_TF32, // 10
			TMAP_DTYPE_MXINT4, // 11
			TMAP_DTYPE_MXFP4, // 12
			TMAP_DTYPE_MXFP6, // 13
			TMAP_DTYPE_MXINT8, // 14
			TMAP_DTYPE_MXFP8 // 15
			};

uint8_t	tensorRank	维度，取值范围为[2,5]	
uint8_t*	globalAddress	device内存地址	linear：64字节对齐 tiled：256字节对齐
uint32_t*	globalDimInElements	以元素为单位的tensor逻辑shape	最低维度（0）为列数 第1维度（1）为行数
uint32_t*	boxDimInElements	以元素为单位的box的逻辑shape	低2维需要被对应维度的tile shape整除 （需要用户来保证，否则driver api报runtime error）
uint32_t	oobFillValue	out-of-region填充	含义参考 SiPU 1.5 DTE 架构设计方案 v1.2 5.2.3章节中的第10部分

1.3 示例代码(SiMemObj)

```cpp
#include <sipu.h>

int main(int argc, char** argv)
{
    SiModule mod;
    SiFunction tcp2tiled;

    auto dev = SiDeviceOpen(0);
    auto subDev = dev->getSubDevice(0);

    auto elf = std::filesystem::path(argv[0]).parent_path() / "tiled_copy.elf";
    SiModuleLoad(mod, elf);
    SiModuleGetFunction(tcp2tiled, mod, "tiled_copy_to_tiled_tensor");

    uint32_t in_size = 2 * 2 * 1024;
    uint32_t out_size = 4 * 4 * 1024;
    uint8_t* host_in = (uint8_t*)malloc(in_size);
    uint8_t* host_out = (uint8_t*)malloc(out_size);

    SiMemObj bo_in;
    SiMemObj bo_out;
    SiMemObj bo_tmap;

    MemFlag flag;
    flag.fields.alignment_n = 8;
    SiMalloc(subDev, bo_in, in_size, flag.flags);
    SiMalloc(subDev, bo_out, out_size, flag.flags);

    /*
    // https://siorigin.feishu.cn/docx/SoN1dFhjrotUKDxrpXJcYgqGnXV
    // show case feature 2-2: Tiled copy to Tiled Tensor
    */
    sipu::SItensorMap tmap;
    uint32_t globalDimInElements[2] = {128, 128};
    uint32_t boxDimInElements[2] = {64, 64};
    auto ret = sipu::siTensorMapEncodeTiled(&tmap,
sipu::SItensorMapDataType::TMAP_DTYPE_UINT8,
                                            2,
                                            (uint8_t*)bo_in->data(),
                                            globalDimInElements,
                                            boxDimInElements,
                                            0);
    if(SIPU_SUCCESS != ret)
    {
        printf("siTensorMapEncodeTiled failed\n");
        return -1;
```

```
    }
    printf("siTensorMapEncodeTiled success\n");

    if(SiMalloc(subDev, bo_tmap, sipu::TMAP_SIZE, flag.flags) != SIPU_SUCCESS)
    {
        printf("Failed to allocate memory for tensormap\n");
        return -1;
    }
    SiMemcpyHtoD(bo_tmap, &tmap, sizeof(sipu::SItensorMap));

    for(int i = 0; i < in_size; i++)
    {
        host_in[i] = (i / 32) % 256;
    }

    SiMemcpyHtoD(bo_in, host_in, in_size);

    void* args[] = {bo_tmap->data(), bo_in->data()};

    dim3 grid{1};
    dim3 cluster{1};
    dim3 block{1};

    SiLaunchKernel(subDev, tcp2tiled, grid, cluster, block, 0, args, 0);

    SiMemcpyDtoH(host_out, bo_out, out_size);

    uint64_t check_sum = 0;
    for(int i = 0; i < out_size; i++)
    {
        check_sum += host_out[i];
    }
    printf("check sum: %lu\n", check_sum);

    if(check_sum != 260096)
    {
        printf("check sum failed\n");
        return -1;
    }

    SiFree(bo_in);
    SiFree(bo_out);
    SiFree(bo_tmap);

    return 0;
}
```

1.4 示例代码(runtime接口)

代码块

```
# build
scc -o build/test test.cpp
# run
./build/test
```

代码块

```
#include <cstdio>

#include "sipu_runtime.h"
#include "sipu.h"

__global__ void get_tmap_sum(void *inTmap, int len, void *inSum)
{
    uint8_t *data = reinterpret_cast<uint8_t *>(inTmap);

    // checksum the data
    uint32_t checksum = 0;
    for (uint32_t i = 0; i < len; i++)
    {
        checksum += data[i];
    }
    *reinterpret_cast<uint32_t *>(inSum) = checksum;
}

int main(int argc, char *argv[])
{
    // device memory for tensor
    // NOTE: this is a dummy tensor data, it will be replaced by the actual tensor data
    void *devTensorData = nullptr;
    sipuMalloc(&devTensorData, 128 * 128 * sizeof(uint8_t));

    // device memory for tensor map
    void *devTmap = nullptr;
    sipuMalloc(&devTmap, sizeof(sipu::SItensorMap));

    // encode tensor map on host
    sipu::SItensorMap tmap;
    uint32_t globalDimInElements[2] = {128, 128};
    uint32_t boxDimInElements[2] = {64, 64};
    auto ret = sipu::siTensorMapEncodeTiled(&tmap,
```

```
35                                                  sipu::SItensorMapDataType::TMAP_DTYPE_UINT8,
36                                                  2,
37                                                  (static_cast<uint8_t *>(devTensorData)),
38                                                  globalDimInElements,
39                                                  boxDimInElements,
40                                                  0);
41      if (SIPU_SUCCESS != ret)
42      {
43          printf("siTensorMapEncodeTiled failed\n");
44          return -1;
45      }
46      printf("siTensorMapEncodeTiled success\n");
47
48      // copy tensor map to device
49      sipuMemcpy(devTmap, &tmap, sizeof(sipu::SItensorMap), sipuMemcpyHostToDevice);
50
51      // use tensor map on device
52      void *devCheckSum = nullptr;
53      sipuMalloc(&devCheckSum, sizeof(uint32_t));
54      get_tmap_sum<<<1, 1, 1>>>(devTmap, sizeof(sipu::SItensorMap), devCheckSum);
55
56      uint32_t sumFromDevice = 0;
57      sipuMemcpy(&sumFromDevice, devCheckSum, sizeof(uint32_t), sipuMemcpyDeviceToHost);
58
59      printf("sum from device: %d\n", sumFromDevice);
60
61      // sum in host
62      uint32_t sum = 0;
63      uint8_t *data = reinterpret_cast<uint8_t *>(&tmap);
64      for (uint32_t i = 0; i < sizeof(sipu::SItensorMap); i++)
65      {
66          sum += data[i];
67      }
68      printf("sum from host: %d\n", sum);
69
70      // free device memory
71      sipuFree(devTensorData);
72      sipuFree(devTmap);
73      sipuFree(devCheckSum);
74
75      return 0;
76  }
```

2. siTensorMapEncodeLinear

代码块

```
/**
 * @brief Encode a linear tensor map
 * @note this function only support tiled tensor map
 * @param tmap Tensor map object to create.
 * @param tensorDataType Tensor data type
 * @param tensorRank  Dimensionality of tensor; range is [2, 5]
 * @param globalAddress Starting address of memory region described by tensor, 256-byte aligned
 * @param globalDimInElements Array containing tensor size (number of elements) along each of the
 * tensorRank dimensions. eg, {512, 512}. Length of the array is tensorRank.
 * @param boxDimInElements Array containing traversal box size (number of elments) along each of the
 * tensorRank dimensions. Specifies how many elements to be traversed along each tensor dimension.
 * eg, {128, 128}. Length of the array is tensorRank.
 * @note the dim[0] and dim[1]'s value of boxDimInElements must be divisible by the tile dimension.
 * @param oobFillValue Value to fill out-of-bound elements
 * @return SIresult
 */
SIresult siTensorMapEncodeLinear(SItensorMap* tmap,
                                 SItensorMapDataType tensorDataType,
                                 uint8_t tensorRank,
                                 uint8_t* globalAddress,
                                 uint32_t* globalDimInElements,
                                 uint32_t* boxDimInElements,
                                 uint32_t oobFillValue);
```

参数说明（同上）

3. 单个参数修改

NOTES：driver API只做数据**透传填充**，用户需自行保证数据有效性。

代码块

```
SIresult siTensorMapReplaceDataType(SItensorMap* tmap, SItensorMapDataType tensorDataType)
{
    tmap->tensor_datatype = tensorDataType;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceRank(SItensorMap* tmap, uint8_t tensorRank)
{
    tmap->tensor_rank = tensorRank;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceTiled(SItensorMap* tmap, uint8_t tiled)
{
    tmap->tensor_tiled = tiled;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceTileDim(SItensorMap* tmap, uint32_t tileDim0, uint32_t tileDim1)
{
    tmap->tile_dim[0] = tileDim0;
    tmap->tile_dim[1] = tileDim1;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceTileHeaderSize(SItensorMap* tmap, uint32_t tileHeaderSize)
{
    tmap->tile_headersize = tileHeaderSize;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceTileDataSize(SItensorMap* tmap, uint32_t tileDataSize)
{
    tmap->tile_datasize = tileDataSize;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceSuperTileDim(SItensorMap* tmap, uint32_t superTileDim0, uint32_t superTileDim1)
{
    tmap->supertile_dim[0] = superTileDim0;
    tmap->supertile_dim[1] = superTileDim1;
    return SIPU_SUCCESS;
```

```
}

SIresult siTensorMapReplaceGlobalAddr(SItensorMap* tmap, uint64_t globalAddr)
{
    tmap->global_addr = globalAddr;
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceGlobalDim(SItensorMap* tmap, uint32_t* globalDim)
{
    constexpr auto rank = sizeof(SItensorMap::global_dim) /
sizeof(SItensorMap::global_dim[0]);
    for(auto i = 0; i < rank; i++)
    {
        tmap->global_dim[i] = globalDim[i];
    }
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceGlobalStrides(SItensorMap* tmap, uint32_t*
globalStrides)
{
    constexpr auto rank = sizeof(SItensorMap::global_strides) /
sizeof(SItensorMap::global_strides[0]);
    for(auto i = 0; i < rank; i++)
    {
        tmap->global_strides[i] = globalStrides[i];
    }
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceBoxDim(SItensorMap* tmap, uint32_t* boxDim)
{
    constexpr auto rank = sizeof(SItensorMap::box_dim) /
sizeof(SItensorMap::box_dim[0]);
    for(auto i = 0; i < rank; i++)
    {
        tmap->box_dim[i] = boxDim[i];
    }
    return SIPU_SUCCESS;
}

SIresult siTensorMapReplaceOobFillValue(SItensorMap* tmap, uint32_t
oobFillValue)
{
    tmap->oob_fillvalue = oobFillValue;
    return SIPU_SUCCESS;
```

```
85  }
```

4. 任意参数构造

为了支持更加灵活的tensormap生成，满足不同的需求，可以通过传递所有tensormap的字段进行tensormap的构造。API会将传入的参数强制替换到通过第一个参数传入tensormap对象中。

> 🧨 小众需求，强烈不推荐使用
>
> NOTES：driver API只做数据**透传填充**，用户需自行保证数据有效性。

代码块

```
1   /**
2    * @brief Encode a tensor map with full fields.
3    * @note This function do not check the input parameters except the null ptr,
    it is the caller's
4    * responsibility to ensure the input parameters are valid.
5    * @param tmap Tensor map object to create.
6    * @param tensorDataType Tensor data type.
7    * @param tensorRank  Dimensionality of tensor; range is [2, 5].
8    * @param tensorTiled Tensor tiled flag.
9    * @param tileDim Tile dimension. Size is 2.
10   * @param tileHeaderSize Tile header size.
11   * @param tileDataSize Tile data size.
12   * @param supertileDim Supertile dimension. Size is 2.
13   * @param globalAddress Global address.
14   * @param globalDim Global dimension. Size is 5.
15   * @param globalStrides Global strides. Size is 5.
16   * @param boxDim Box dimension. Size is 5.
17   * @param oobFillValue Out-of-bound fill value.
18   * @return SIresult
19   */
20  SIresult siTensorMapEncode(SItensorMap* tmap,
21                             SItensorMapDataType tensorDataType,
22                             uint8_t tensorRank,
23                             uint8_t tensorTiled,
24                             uint32_t* tileDim,
25                             uint32_t tileHeaderSize,
26                             uint32_t tileDataSize,
27                             uint8_t* supertileDim,
28                             uint8_t* globalAddress,
29                             uint32_t* globalDim,
```

```
                            uint32_t* globalStrides,
                            uint32_t* boxDim,
                            uint32_t oobFillValue);
```

Sample（gcc编译器）

代码块

```
#include "sipu_runtime.h"
#include "sipu.h"

int main(int argc, char *argv[])
{
    sipu::SItensorMap tmap;

    auto tensorDataType = sipu::SItensorMapDataType::TMAP_DTYPE_FP16;
    auto tensorRank = 2;
    auto tensorTiled = 1;
    auto tileDim = std::array<uint32_t, 2>{16, 32};
    auto tileHeaderSize = 0;
    auto tileDataSize = 1024;
    auto supertileDim = std::array<uint8_t, 2>{1, 1};
    auto globalAddress = 0x10000000;
    auto globalDim = std::array<uint32_t, 5>{2, 2, 1, 1, 1};
    auto globalStrides = std::array<uint32_t, 5>{2, 2, 1, 1, 1};
    auto boxDim = std::array<uint32_t, 5>{1, 1, 1, 1, 1};
    auto oobFillValue = 0x00;

    auto ret = sipu::siTensorMapEncode(&tmap,
                                       tensorDataType,
                                       tensorRank,
                                       tensorTiled,
                                       tileDim.data(),
                                       tileHeaderSize,
                                       tileDataSize,
                                       supertileDim.data(),
                                       reinterpret_cast<uint8_t *>
(globalAddress),
                                       globalDim.data(),
                                       globalStrides.data(),
                                       boxDim.data(),
                                       oobFillValue);
    if (ret != SIresult::SIPU_SUCCESS)
    {
        printf("siTensorMapEncode failed\n");
        abort();
```

```
    }
    return 0;
}
```

run_sample.sh

代码块

```
SIPU_SDK_PATH=/share_data/sipu_sdk_debug/sipu_sdk_250722/sipu_sdk_rel
source $SIPU_SDK_PATH/sipu_sdk_setup.sh

g++ --std=c++20 -I $SIPU_SDK_PATH/include -L $SIPU_SDK_PATH/lib -o
encode_from_all encode_from_all.cpp -lsipu -lsipurt -v

./encode_from_all
```